

Lecture 4: Computational Complexity: O , Ω , Θ , and Merge Sort's Recurrence

Zhentao Shen

zhentao.shen@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Computational complexity concerns how resource use grows with input size. Sets specify possible inputs, functions associate inputs with outputs, and quantifiers make growth bounds precise. Big O gives an eventual upper bound, big Ω a lower bound, and big Θ a matching pair. Merge sort divides an array into balanced halves and merges their sorted results. Summing work across its recursion levels gives a tight time bound of $\Theta(n \log n)$. Counting only comparisons gives a separate, exact worst-case recurrence.

AI use declaration. We used GPT-6 to research as well as polish the writing in the notes.
--

1 Motivation

A researcher may need to sort hospital visits by arrival time before constructing a chronological record. The procedure may eventually process millions of visits. The question is how operation counts grow with record counts.

Background. This chapter assumes elementary algebra and comparisons of numbers. Sets, functions, and growth notation are defined here. The hospital example is illustrative. No clinical data are used.

2 Setup and Notation

2.1 Sets, Quantifiers, and Functions

Sets and membership. A *set* is a collection of distinct elements. For example, $S := \{2, 4, 6\}$ contains three elements. The symbol $:=$ means “is defined as,” $x \in S$ means that x belongs to S , and $|S|$ denotes its cardinality, or number of elements when S is finite. Order and repetition are irrelevant: $\{2, 4, 2\} = \{4, 2\}$.

For sets S and U , the relation $S \subseteq U$ means every element of S belongs to U . The relation $S \subsetneq U$ additionally requires $S \neq U$. We use the following number sets:

$$\begin{aligned}\mathbb{N} &:= \{0, 1, 2, \dots\}, & \mathbb{Z} &:= \{\dots, -2, -1, 0, 1, 2, \dots\}, \\ \mathbb{Q} &:= \{p/q : p \in \mathbb{Z}, q \in \mathbb{N}, q > 0\}, & \mathbb{R} &:= \text{the real numbers}.\end{aligned}$$

Thus $\mathbb{N} \subsetneq \mathbb{Z} \subsetneq \mathbb{Q} \subsetneq \mathbb{R}$. The values -1 , $1/2$, and $\sqrt{2}$ establish these strict inclusions.

Quantifiers. The symbol $\exists$ means “there exists,” and $\forall$ means “for every.” For example, $\forall x \in \mathbb{N} \exists y \in \mathbb{N} (y > x)$: for each x , choose $y = x + 1$. Reversing these quantifiers gives $\exists y \in \mathbb{N} \forall x \in \mathbb{N} (y > x)$, which is false. No single natural number exceeds all natural numbers. Growth bounds likewise require fixed constants.

Functions. Assigning each visit exactly one arrival time is an example of a function. A *function* $f : S \rightarrow U$ assigns each element x of its *domain* S exactly one value $f(x)$ in its *codomain* U . Its *image* is the set $f(S) := \{f(x) : x \in S\}$. Different inputs may have the same output, and some codomain elements may never occur as outputs (Figure 1).

One-to-one correspondence. A function is *injective* when distinct inputs have distinct outputs. It is *surjective* when every codomain element occurs as an output. A *bijection* has both properties. Equal cardinality means a bijection exists. A set in bijection with $\mathbb{N}$ is *countably infinite*.

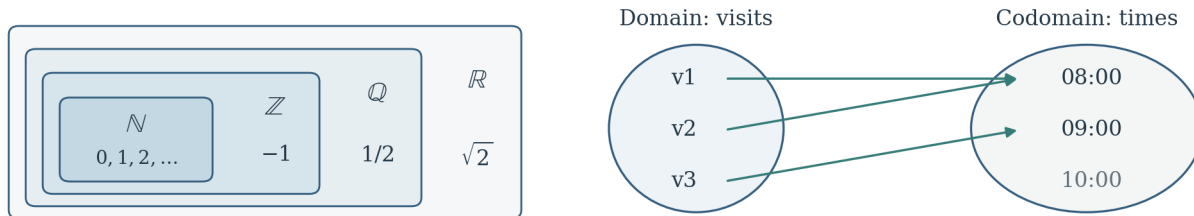


Figure 1: Nested number sets and a function from visits to arrival times. Two visits share an output. 10:00 belongs to the codomain but not the image.

2.2 From Mathematical Inputs to Computational Cost

Quiz: unique labels for rational numbers. *True or False? $\forall x \in \mathbb{Q}, \exists y \in \mathbb{N}$ such that y is a unique label for x .*

The answer is *True*, provided “unique label” means that different rationals receive different labels. Formally, we seek an injective function $h : \mathbb{Q} \rightarrow \mathbb{N}$. Merely requiring one label per input is insufficient: the assignment $h(x) = 0$ gives every input one value but distinguishes none.

Construction and equivalence. Start a list with 0. Visit positive integer pairs (p, q) by increasing $p + q$, and then by increasing p within each diagonal. Append p/q and $-p/q$, skipping previously listed values. The list begins

$$0, 1, -1, \frac{1}{2}, -\frac{1}{2}, 2, -2, \frac{1}{3}, -\frac{1}{3}, \dots$$

Any nonzero rational has a representation p/q or $-p/q$ with positive integers p, q . Its diagonal has finite index $p + q$, so it is reached after finitely many steps. Duplicate removal, for example skipping $2/2$ after $1/1$, gives exactly one occurrence. Assign each rational its position, starting at zero. This yields a bijection, proving $|\mathbb{Q}| = |\mathbb{N}|$.

An injective labeling need not use every natural number. Nevertheless, because $\mathbb{Q}$ is infinite, its used labels can be listed in increasing order and renumbered $0, 1, 2, \dots$ to obtain a bijection. Thus the quiz, interpreted as *distinct* natural-number labels, is equivalent to the cardinality claim (Lehman et al., 2018, Section 8.1).

Arrays preserve positions. An *array* is an indexed sequence that preserves order and repeated values. Let A have length n , with entries $A[0], \dots, A[n-1]$. For integers $0 \leq i \leq j \leq n$, the half-open slice $A[i : j]$ contains indices i through $j-1$. It has length $j-i$ and is empty when $i = j$. We write $A[:j]$ for $A[0 : j]$ and $A[i:]$ for $A[i : n]$.

A cost function. Let $\mathcal{I}_n$ contain the allowed length- n inputs to a fixed algorithm, and let $t(A)$ count elementary operations. Assuming a finite maximum, define the *worst-case* cost by

$$T(n) := \max_{A \in \mathcal{I}_n} t(A).$$

Best-case cost uses a minimum. Average-case cost requires an input distribution. We assume constant-time comparisons, array access, and record-reference copies for fixed-size keys. Operation counts do not measure seconds.

3 Main Development

3.1 Asymptotic Upper, Lower, and Tight Bounds

An eventual comparison. Suppose an operation count is $T(n) = 3n + 2$. For every integer $n \geq 2$, $3n \leq T(n) \leq 4n$. Constant multiples of n therefore bound this cost from both sides, even though $T(n)$ is not equal to n .

Definition 1 (Asymptotic bounds). Let $T, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ be nonnegative functions, where $\mathbb{R}_{\geq 0}$ denotes the nonnegative real numbers. Assume $g(n) > 0$ for all sufficiently large n .

- $T \in O(g)$ if there exist $c > 0$ and $n_0 \in \mathbb{N}$ such that $T(n) \leq c g(n)$ for every integer $n \geq n_0$.
- $T \in \Omega(g)$ if there exist $c > 0$ and $n_0 \in \mathbb{N}$ such that $c g(n) \leq T(n)$ for every integer $n \geq n_0$.
- $T \in \Theta(g)$ if there exist $c_1, c_2 > 0$ and $n_0 \in \mathbb{N}$ such that $c_1 g(n) \leq T(n) \leq c_2 g(n)$ for every integer $n \geq n_0$.

Constants are independent of n . The notation $T(n) = O(g(n))$ means class membership, not ordinary equality. A tight bound requires upper and lower bounds. The larger threshold ensures that both hold (Lehman et al., 2018, Section 14.7).

For $3n + 2$, choose $g(n) = n$, $c_1 = 3$, $c_2 = 4$, and $n_0 = 2$. Thus $3n + 2 \in \Theta(n)$ (Figure 2). It also belongs to $O(n^2)$, but that upper bound is not tight. It does not belong to $\Theta(n^2)$ because $(3n + 2)/n^2$ tends to zero.

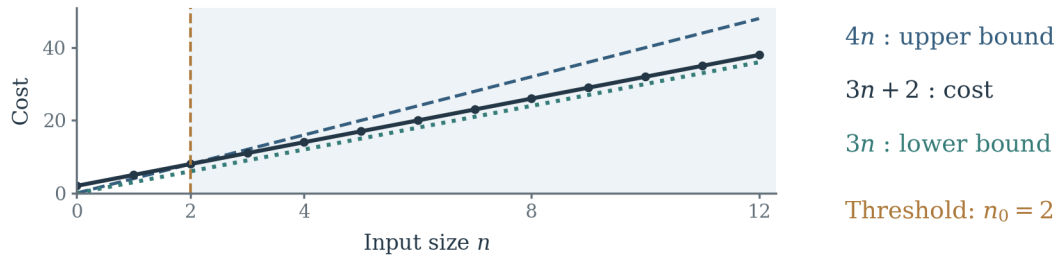


Figure 2: For integer inputs $n \geq 2$, the function $3n + 2$ lies between $3n$ and $4n$. Lines connect integer values only as a visual guide.

Bounds and cases are different choices. The symbols O and Ω do not mean worst case and best case. Either symbol can describe either cost function. Here, T already denotes worst-case cost. Proving $T \in \Omega(g)$ supplies a lower bound on that worst-case function.

3.2 Merge Sort and the Cost of Merging

A concrete merge. Two sorted sequences of arrival-time ranks, $[1, 4]$ and $[2, 3]$, can be combined by comparing their first remaining entries. The comparisons $(1, 2)$, $(4, 2)$, and $(4, 3)$ append 1, 2, and 3 in that order. After the second sequence becomes empty, appending the remaining 4 requires no further key comparison.

The merge procedure. Let L and R be sorted arrays of positive lengths a and b . To *merge* them, repeatedly append the smaller first remaining entry to a new output array. Break ties by taking the entry from L . Once either array is exhausted, append the other array's remaining entries. Each appended entry is a smallest remaining entry, so the output stays sorted. Every entry is appended exactly once.

Each comparison removes one entry from further consideration. After $a + b - 1$ removals, at most one entry remains. Thus merging uses at most $a + b - 1$ key comparisons. Interleaved inputs can attain this bound. Regardless of the comparison count, writing all $a + b$ output entries takes linear time under the stated cost model. The total merge cost is therefore $\Theta(a + b)$.

Recursive sorting. An array is *sorted* if $A[i] \leq A[i + 1]$ whenever $0 \leq i < n - 1$. Merge sort recursively sorts two balanced halves, then merges their results. Here $\lfloor x \rfloor$ is the greatest integer at most x .

Algorithm 1 Merge sort with half-open slices

```
1: function MERGESORT( $A$ )
2:    $n \leftarrow \text{length}(A)$ 
3:   if  $n \leq 1$  then
4:     return  $A$ 
5:   end if
6:    $m \leftarrow \lfloor n/2 \rfloor$ 
7:    $L \leftarrow \text{MERGESORT}(A[:m])$ 
8:    $R \leftarrow \text{MERGESORT}(A[m:])$ 
9:   return  $\text{MERGE}(L, R)$ 
10: end function
```

Correctness and termination. Arrays of length zero or one are already sorted. For $n > 1$, both recursive inputs are smaller than n . Assuming the recursive calls correctly sort smaller arrays, the merge argument establishes correctness at size n . This is induction on input length. Repeated halving reaches the base case, so the recursion terminates. The standard divide-and-merge analysis is developed in [Erickson \(2019, Sections 1.4 and 1.7\)](#).

3.3 Solving the Time Recurrence

Work across levels. A *recurrence* expresses a cost using costs for smaller inputs. For $n = 2^k$, where k is a nonnegative integer, merge sort produces two subproblems of size $n/2$. Its worst-case time satisfies

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

The additional term covers merging and other work outside the recursive calls. Copying array slices, if required, also takes only linear time per call.

Define $\log_2 n$ as the exponent k such that $2^k = n$. At depth j of the recursion tree there are 2^j subproblems, each of size $n/2^j$. For $0 \leq j < k$, the combined work at that depth is

$$2^j \Theta(n/2^j) = \Theta(n).$$

These constants apply uniformly. There are $k = \log_2 n$ merging levels and n constant-cost leaves. Consequently,

$$T(n) = \Theta(n \log_2 n) + \Theta(n) = \Theta(n \log n). \quad (1)$$

The logarithm base does not affect this bound: for any fixed base $b > 1$, $\log_b n = \log_2 n / \log_2 b$, a constant rescaling.

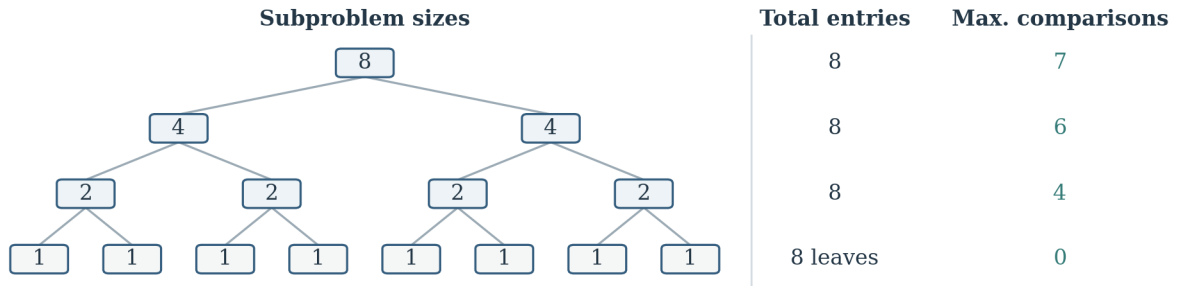


Figure 3: A recursion tree for eight inputs. Internal levels each process eight entries in total. Their worst-case key comparison counts are 7, 6, and 4. Linear-time work and exact comparison counts are distinct quantities.

Sizes that are not powers of two. Let $\lceil x \rceil$ denote the least integer at least x . For arbitrary $n > 1$, the recurrence is

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n).$$

The tree has at most $\lceil \log_2 n \rceil$ merging levels, each with at most linear work. Its first $\lfloor \log_2 n \rfloor$ levels contain only internal nodes, whose subarrays together have length n . Those levels each require at least linear work. Both bounds in Equation (1) therefore hold for general n .

4 Worked Example

4.1 Exact Comparison Counts

A separate cost measure. Let $C(n)$ denote the maximum number of *key comparisons* made by merge sort on n distinct keys. A singleton requires no comparison, so $C(1) = 0$. For powers of two,

$$C(n) = 2C(n/2) + n - 1.$$

To attain the bound, assign alternating ranks to the halves and recursively arrange each half for its worst case. The sorted halves interleave.

At depth j , the 2^j merges each require at most $n/2^j - 1$ comparisons. For $n = 2^k$, summing these attainable counts gives

$$C(n) = \sum_{j=0}^{k-1} (n - 2^j) = nk - (2^k - 1) = n \log_2 n - n + 1. \quad (2)$$

Here $\sum$ means summation over the indicated indices. The geometric sum is $1 + 2 + \dots + 2^{k-1} = 2^k - 1$.

Eight arrival-time ranks. Take $A = [1, 5, 3, 7, 2, 6, 4, 8]$. Sorting the left half merges $[1, 5]$ with $[3, 7]$ in three comparisons. Sorting the right half merges $[2, 6]$ with $[4, 8]$ in three comparisons. Each initial pair needs one comparison. The final merge compares $(1, 2)$, $(3, 2)$, $(3, 4)$, $(5, 4)$, $(5, 6)$, $(7, 6)$, and $(7, 8)$, then appends 8 without a comparison. Thus $C(8) = 4 + 6 + 7 = 17$, agreeing with Equation (2). This excludes copying, indexing, and function calls.

Merge output size	Number of merges	Comparisons per merge	Total
2	4	1	4
4	2	3	6
8	1	7	7

Table 1: Comparison counts for the eight-rank example.

5 Summary

- Sets contain distinct elements. Arrays preserve order and repetitions. Functions assign one output to each input.
- Every rational number can receive a distinct natural-number label. Diagonal enumeration with duplicate removal proves $|\mathbb{Q}| = |\mathbb{N}|$.
- Big O , Ω , and Θ express upper, lower, and tight growth bounds using fixed constants beyond a threshold.
- Standard merge sort takes $\Theta(n \log n)$ worst-case time. For power-of-two sizes, the exact worst-case comparison count is $n \log_2 n - n + 1$.

Stability and storage. Two hospital visits may share an arrival time. A *stable* sort preserves their original relative order. Taking the left entry on a tie preserves that order during merging. Applying this rule at every level makes merge sort stable.

An array implementation can reuse an auxiliary buffer of n entries, requiring $\Theta(n)$ additional storage. The nested calls use $O(\log n)$ stack space, where the stack stores the state of unfinished calls. These are space bounds, not time bounds. The buffer permits linear-time merging but adds a memory cost that matters for large datasets. This bound assumes sequential execution and reuse or release of temporary storage.

References

- Erickson, J. (2019). *Algorithms*. Self-published, first edition.
- Lehman, E., Leighton, F. T., and Meyer, A. R. (2018). *Mathematics for Computer Science*. Massachusetts Institute of Technology. Revised June 6, 2018.